

#WHAT IS AI?

- AI takes the advantages of the computers, and machines to mimic the problem solving, and decision making capabilities of the human beings.
- An AI system must be rational, to do the right thing.
- A rational AI is one that consistently makes decisions or takes actions that are optimal or near-optimal for achieving its set objectives, based on the data and rules it has access to.
- Definition of AI are categorized into 4 categories:

(i) System that thinks like humans

(ii) Systems that think rationally

(iii) Systems that act like humans

(iv) Systems that act rationally

★ A human-centred approach is an empirical science, involving hypothesis and experimental observations

★ A rationalist approach involves a combination of mathematics and engineering.

Rationalist Approach = Mathematics + Engineering

→ It is internal thought processes and reasoning

→ Practical intelligence solves real-world problems

- Ethical intelligence promotes responsible use of abilities

- Consistent intelligence means doing the right thing repeatedly.

① Acting Humanly: Turing Test

- The Turing Test, proposed by Alan Turing (1950), was designed to provide a satisfactory operational definition of intelligence.
- A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses came from a person or from a computer.
- The Turing Machine (Computer) would need to possess the following human capabilities:
 - (i) Natural Language Processing (NLP) - To communicate
 - (ii) Knowledge Representation - to store info provided before or during the interrogation.
 - (iii) Automated Reasoning - to use the stored info to answer questions and to draw new conclusions.
 - (iv) Machine Learning (ML) - to adapt to new circumstances and to detect and extrapolate patterns.
- no physical contact.
- Total Turing Test:
 - Includes a video signal so that the interrogator can test the subject's perceptual abilities, as well as the opportunity for the interrogator to pass physical objects through the hatch.
- To pass the total Turing Test, the computer will need:
 - (i) Computer Vision to perceive objects.
 - (ii) robotics to manipulate objects and move about.

② Thinking humanly: The cognitive modelling approach

- Introspection: trying to catch our thoughts as they go by
 - ↳ through psychological experiments
 - ↳ observing a person in action
 - ↳ brain imaging
- cognitive science combines AI computer models and psychological experiments to develop testable theories about the human mind.

③ Thinking rationally: The "laws of thought" approach

- the Greek philosopher Aristotle was one of the first to attempt to codify "right thinking", that is, irrefutable reasoning processes.
- His syllogisms provided patterns for argument structures that always yielded correct conclusions when given correct premises.
- These laws of thought were supposed to govern the operation of the mind; their study initiated the field called logic.
- Thinking rationally involves using logic, evidence, and structured reasoning to make decisions or form beliefs in a way that is objective, consistent, and free from emotional bias.
- Rational thinking is typically characterized by several core principles:
 - (i) Logic & reason
 - (ii) Evidence-Based
 - (iii) Clarity and Precision
 - (iv) Consistency

→ Obstacles:

- (i) it is not easy to take informal knowledge and state it in the formal terms required by logical notation, particularly when the knowledge is less than 100% certain.

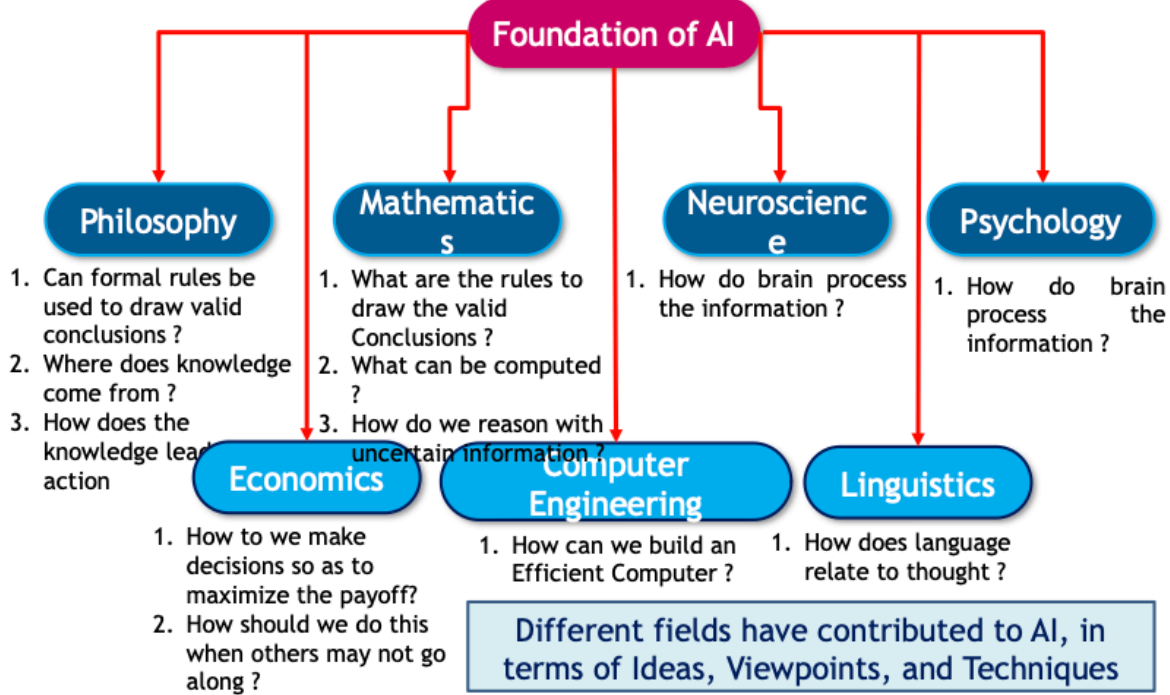
(ii) there is a big difference b/w solving a problem "in principle" and solving it "in practice".

④ Acting Rationally : The Rational Agent Approach:

- An agent is just something that acts
- Computer agents operate autonomously, perceive their environment, persist over time, adapt, and pursue goals.
- A rational agent seeks that best outcome or the best expected outcome under uncertainty.
- the "laws of thought" in AI focus on correct inferences.
- to act rationally, deduce the best action, and act on it.
- skills for the Turing test enable rational action.
- limited rationality - Acting appropriately when there is not enough time to do all the computations one might like.
- Advantages:
 - (i) it is more general than the "laws of thought" approach because correct inference is just one of several possible mechanisms for achieving rationality
 - (ii) it is more amenable to scientific development than are approaches based on human behavior or human thought.



The Foundation of AI



INTELLIGENT AGENTS:

-An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through effectors/actuators.

- A **Human Agent** has eyes, ears, and other organs for sensors, and hands, legs, mouth, and other body parts for effectors
- A **Robotic Agent** substitutes cameras and infrared range finders for the sensors and various motors for the effectors.
- A generic agent is diagrammed in shown in below Figure.

- Examples for Agents
 - Human Agent
 - Robotic Agent

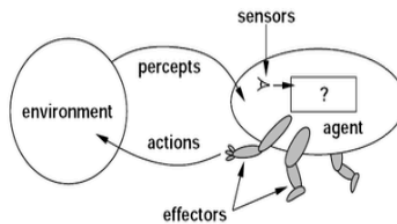
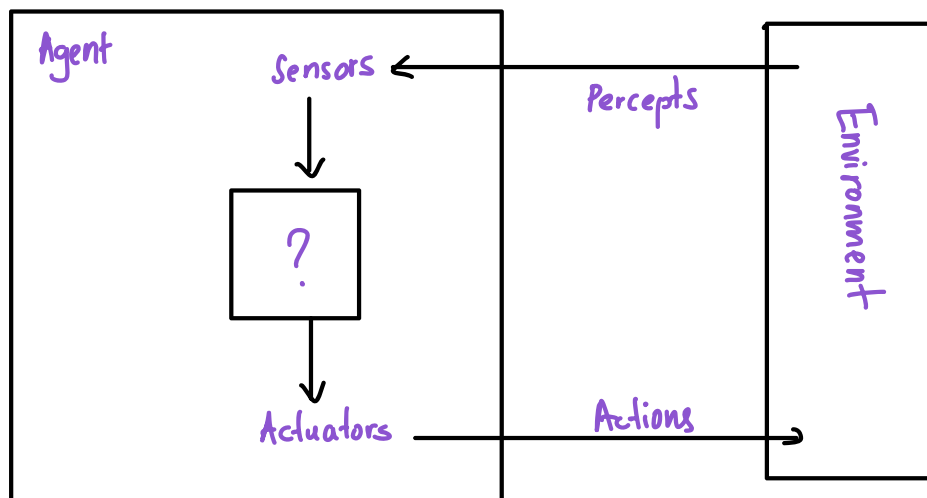


Figure. Agents interact with Environments through Sensors and Effectors

33



→ Agent :

- X-thing, which perceives the environment through sensors and acts on the environment through actuators.
- Human agent/software agent/ Robotic agent.

→ Percept:

- Is what the agent senses from environment.

→ Percept Sequence:

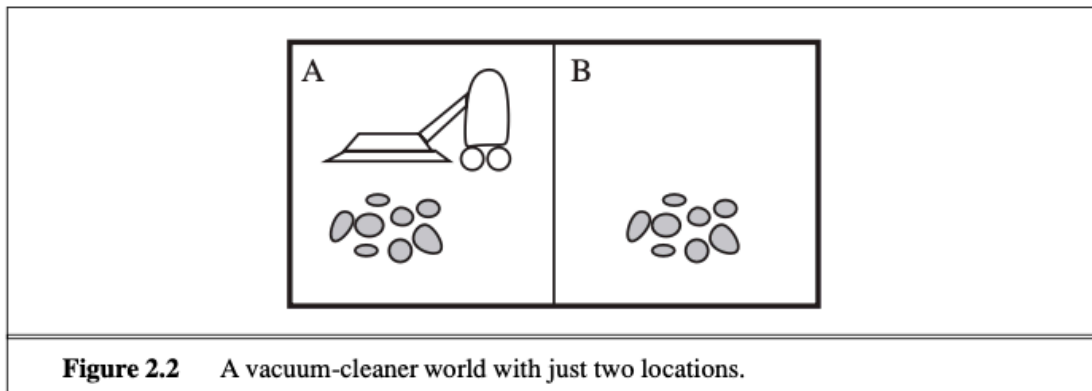
- Complete history of sensing

→ Agent Function:

- Maps percept sequence to action

→ Agent Program:

- Is the implementation of the agent function.



Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

#GOOD BEHAVIOR: THE CONCEPT OF RATIONALITY:

- a rational agent is one that does the right thing - conceptually speaking, every entry in the table for the agent function is filled out correctly.

— Performance Measures:

- Consequentialism: evaluating agent's behaviour by its consequence
- Agents generates sequence of actions based on the percept
- the sequence of actions causes the sequence of states in agents environment.
- If the sequence is desirable, then the agent has performed well

↳ captured by performance measure

- this notion of desirability is captured by a performance measure that evaluates any given sequence of environment states.

★ As a general rule, it is better to design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave.

⇒ Rationality:

What is rational at any given time depends on 4 things:

- (i) the performance measure that defines the criterion of success
- (ii) The agent's prior knowledge of the environment
- (iii) the actions that the agent can perform
- (iv) The agent's percept sequence to date.

- Definition of a rational agent:

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

✦ An omniscient agent knows the actual outcome of its actions and can act accordingly, but omniscience is impossible in reality.

THE NATURE OF ENVIRONMENTS:

- Specifying the task environment

↳ $P \in A S$ (Performance, Environment, Actuators, Sensors)

- Before designing the agent, the PEAS description is the first step.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

- **Agent:** Social Media Recommendation System
- **Task:** Maximize user engagement
- **Performance:** Time spent on platform, clicks, likes, shares
- **Environment:** Users, their devices, content feeds, user behavior
- **Actuators:** Content ranking, feed updates, push notifications
- **Sensors:** User interaction logs, click-through rates, scrolling behavior

⇒ Properties of Task Environment:

Property	Description	Example (Soccer Robot)
Observable	Is the agent able to perceive the entire environment at each moment?	<i>Partially observable</i> —may not see behind obstacles
Agents	Is the environment single-agent or multi-agent?	<i>Multi-agent</i> —multiple robots and opponents
Deterministic	Does the next state depend only on the current state and action?	<i>Stochastic</i> —ball may bounce unpredictably
Episodic	Are experiences divided into independent episodes?	<i>Sequential</i> —actions affect future outcomes
Static vs Dynamic	Does the environment change while the agent is deliberating?	<i>Dynamic</i> —other players move during decision-making
Discrete vs Continuous	Are states and actions countable or infinite?	<i>Continuous</i> —positions and movements vary smoothly
Known vs Unknown	Does the agent know the rules and outcomes of actions in advance?	<i>Known</i> —rules of soccer are predefined

Property	Description	Example
Fully Observable	Agent has complete information	Chess
Partially Observable	Agent has incomplete information	Self-driving cars
Deterministic	Actions lead to predictable outcomes	Solving a puzzle
Stochastic	Outcomes are partly random	Weather prediction
Episodic	Each action is independent of previous actions	Image recognition
Sequential	Actions have long-term impact	Chess
Static	Environment does not change	Crossword puzzle
Dynamic	Environment changes over time	Traffic navigation
Discrete	Finite number of states or actions	Board games
Continuous	Infinite possibilities for states or actions	Driving
Single-Agent	Only one decision-maker	Maze-solving
Multi-Agent	Multiple decision-makers, cooperative or competitive	Soccer game
Known	Environment's rules and dynamics are understood	Board games
Unknown	Environment is not fully understood; agent must explore	Unfamiliar terrain

★ The hardest case is : partially observable , multi agent , stochastic , sequential , dynamic , continuous , and unknown .

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

STRUCTURE OF THE AGENT:

Agent = Architecture + Program

function TABLE-DRIVEN-AGENT(*percept*) **returns** an action

persistent: *percepts*, a sequence, initially empty

table, a table of actions, indexed by percept sequences, initially fully specified

append *percept* to the end of *percepts*

action $\leftarrow$ LOOKUP(*percepts*, *table*)

return *action*

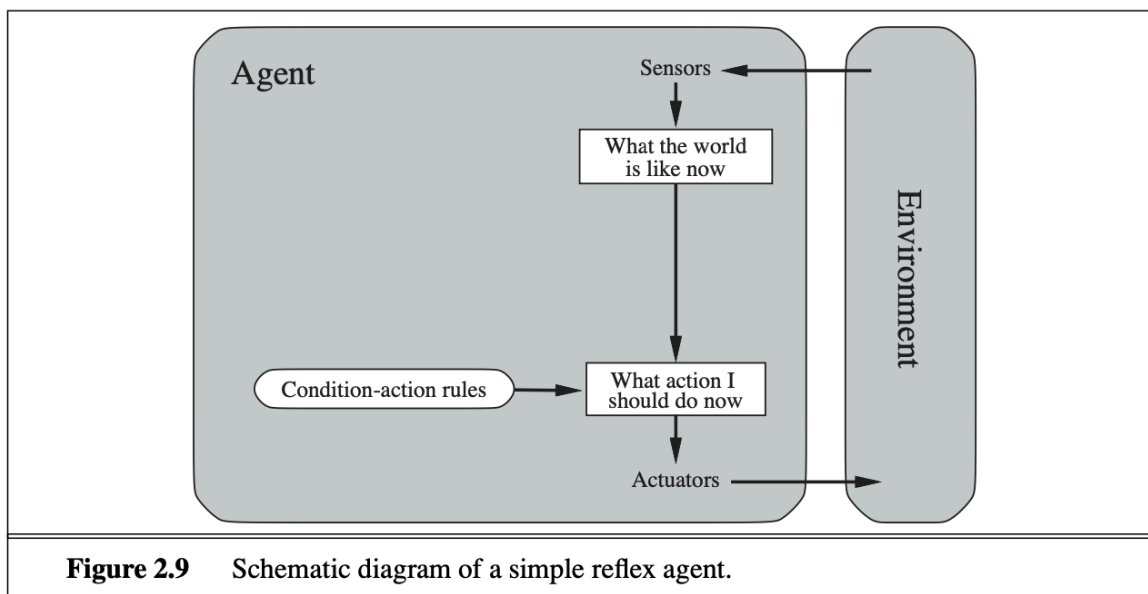
Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

★ As the number of potential percepts and the agent's lifespan expand, the size of the lookup table increases exponentially and becomes impractical.

→ Types of agent programs:

① Simple - Reflex Agent:

- these agents select actions on the basis of the current percept, ignoring the rest of the percept history.
- no memory
- no learning or adaptation
- Use condition-action rules
- Simple and fast
- cannot handle partially observable environments



function SIMPLE-REFLEX-AGENT(*percept*) **returns** an action
persistent: *rules*, a set of condition–action rules

```

state ← INTERPRET-INPUT(percept)
rule ← RULE-MATCH(state, rules)
action ← rule.ACTION
return action

```

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

- Limitation: the details of how models and states are represented widely depends on the type of environment and the particular technology used in the agent design.

② Model-Based Agents:

Properties:

- (i) Internal model: the agent maintains an internal model of the environment. This model represents knowledge about how the environment behaves, including how the agent's actions impact the environment.
- (ii) State tracking: The agent uses its model to track the current state, even if it cannot directly observe everything.

- (iii) Adaptable to partially observable environments: the model helps fill in gaps where direct perception is limited.
- (iv) Limited Memory and Reasoning: Although they use a model, these agents are generally not design for extensive planning or future prediction. They work best in environments where immediate, short-term info is sufficient for making decisions

Simple Reflex Agent vs Model-based

Condition (State)	Action
AmbientLightLevel = Low	TurnOnHeadlights
ObstacleDetected = True AND VehicleSpeed > 60 km/h	AlertDriver
DriverAlertnessLevel = Low	SoundAlert / Suggest Break
AmbientLightLevel = High	TurnOffHeadlights

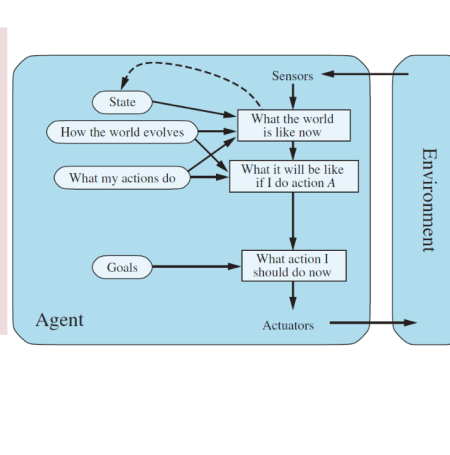
Condition (State + Model)	Action
AmbientLightLevel = Low AND WeatherCondition = Foggy	ReduceSpeed
ObstacleDetected = True AND LanePosition = Center	MaintainLane & Brake
LanePosition = OffCenter AND NoObstacleDetected	AdjustSteering
Speed > SafeLimit AND RoadModel = Curvy	AlertDriver / SlowDown

③ GOAL BASED REFLEX AGENTS:

In addition to a description of the current state, the agent requires goal information, which outlines the situations that are considered desirable.

**Reaching a goal may be a one-step action
OR**

It requires searching the state space and Planning too



— Properties:

- (i) Goal-oriented
- (ii) Decision-Making Based on Goals
- (iii) Planning Capabilities

↳ use planning capabilities, such as search algorithms, to determine the best sequence of actions to reach the goal.

(iv) Flexible and adaptable: these agents can adapt their behavior as conditions change in the environment.

(v) Requires Computational Resources: Goal-based agents generally require more processing power and memory than simple or model-based reflex agents, as they must assess possible outcomes and plan actions accordingly.

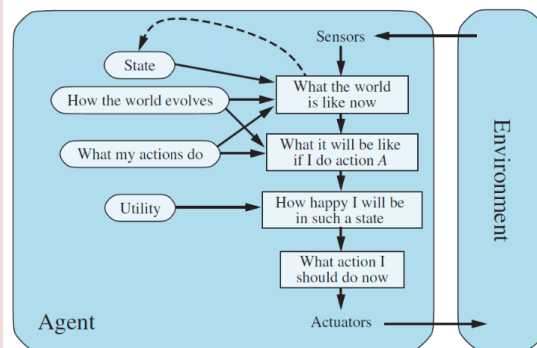
④ Utility-based reflex Agents:

Utility is a measurement of the happiness state of an agent.

It has flexibility and learning.

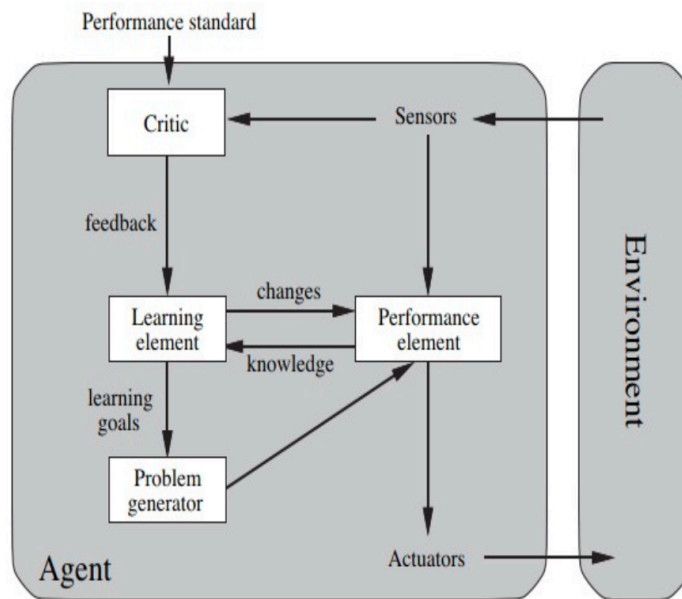
Deals with conflicting goals

It weighs success over goals (sometimes)



⑤ Learning Agents:

- a learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- it starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly 4 conceptual components, which are:
 - (i) Learning elements
 - (ii) Critics
 - (iii) Performance element
 - (iv) Problem Generator.



— Properties:

- (i) Self-improving
- (ii) Exploration and Exploitation
- (iii) Adaptive to changing Environments
- (iv) Uncertainty handling

eg:

- Personalized recommendation system
- Autonomous robots
- self driving Cars

— Limitations:

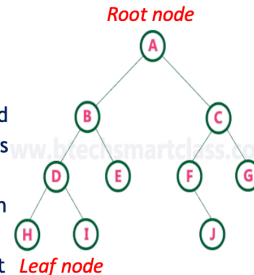
- Data dependence
- Computational demands
- Risk of overfitting

PROBLEM-SOLVING AGENTS:

- rational agents or problem-solving agents in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result.
- problem-solving agents are goal-based agents

Terminologies used in Search Algorithms

- **Search:** Searching is a step by step procedure to solve a search-problem
- **Search Tree:** A tree representation of search problem is called **Search tree**. The root of the search tree is the **root node** which is corresponding to the **Initial state**
- **Path Cost:** It is a function which assigns a numeric cost to each path
- **Solution:** It is an action sequence which leads from the start node to the goal node
- **Optimal Solution:** If a solution has the lowest cost among all solution

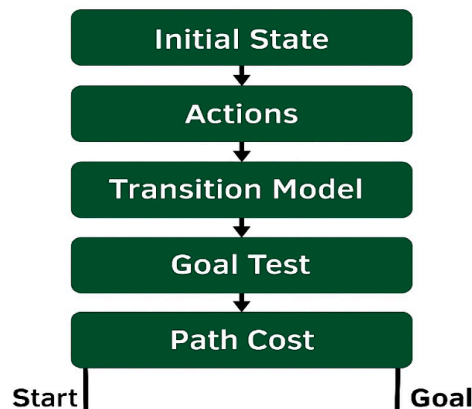


83

Problem-Solving

Goal formulation: Organizes steps to articulate a goal.

Problem formulation: Decides actions to meet the goal.



Example Problems

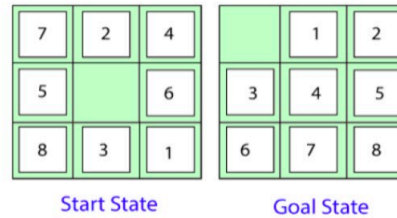
Basically, there are two types of problem approaches:

- Toy Problem: It is a concise and exact description of the problem which is used by the researchers to compare the performance of algorithms.
- Real-world Problem: It is real-world based problems which require solutions. Unlike a toy problem, it does not depend on descriptions, but we can have a general formulation of the problem.

Toy Problems

8 Puzzle Problem: Here, we have a 3×3 matrix with movable tiles numbered from 1 to 8 with a blank space. The tile adjacent to the blank space can slide into that space.

- In the figure, our task is to convert the current state into goal state by sliding digits into the blank space.



Problem-Solving Agents

- **States:** It describes the location of each numbered tiles and the blank tile.
- **Initial State:** We can start from any state as the initial state.
- **Actions:** Here, actions of the blank space is defined, i.e., either left, right, up or down
- **Transition Model:** It returns the resulting state as per the given state and actions.
- **Goal test:** It identifies whether we have reached the correct goal- state.
- **Path cost:** The path cost is the number of steps in the path where the cost of each step is 1.

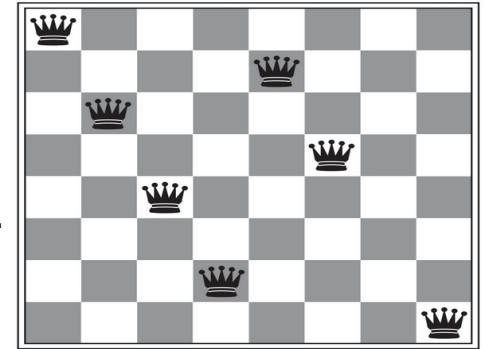
Note: The 8-puzzle problem is a type of sliding-block problem which is used for testing new search algorithms in artificial intelligence.

8-queens problem

For this problem, there are two main kinds of formulation:

Incremental formulation:

1. States: Arrangement of any 0 to 8 queens on the chessboard.
2. Initial State: An empty chessboard
3. Actions: Add a queen to any empty box
4. Transition model: Returns the chessboard with the queen added in a box.
5. Goal test: Checks whether 8-queens are placed on the chessboard without any attack.
6. Path cost: There is no need for path cost because only final states are counted.



Complete-state formulation: It starts with all the 8- queens on the chessboard and moves them around, saving from the attacks.

- Following steps are involved in this formulation

1. States: Arrangement of all the 8 queens one per column with no queen attacking the other queen.
2. Actions: Move the queen at the location where it is safe from the attacks.

Search Strategies (Contd.)

→ Properties of Search Algorithms

- **Completeness:** A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.
- **Optimality:** If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution.
- **Time Complexity:** Time complexity is a measure of time for an algorithm to complete its task.
- **Space Complexity:** It is the maximum storage space required at any point during the search, as the complexity of the problem.

Search Strategies (Contd.)

→ Uniformed Search

- The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.
- It is also called *Blind search*. It examines each node of the tree until it achieves the goal node.

→ Informed Search

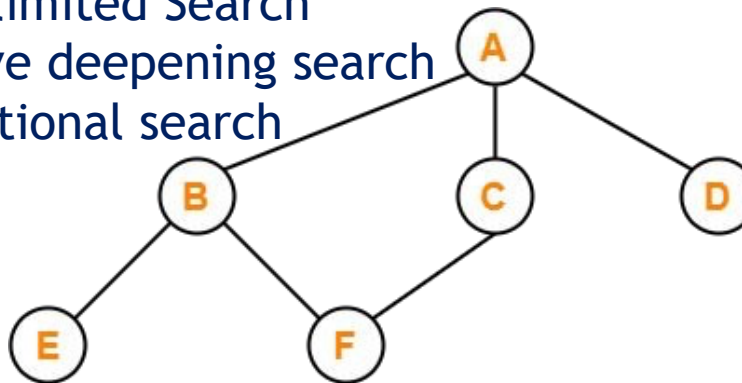
- Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search.
- Informed search strategies can find a solution more efficiently than an uninformed search strategy.
- Informed search is also called a *Heuristic Search*

Search Strategies (Contd.)

Search Strategies

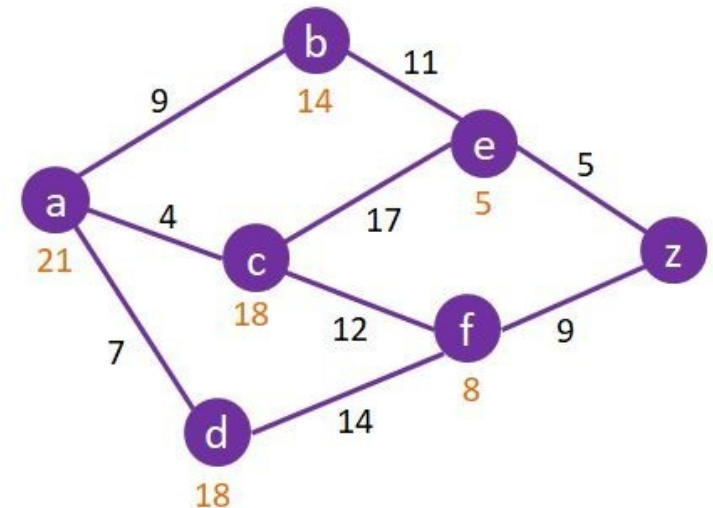
Uninformed Strategies

1. Breadth First Search
2. Uniform Cost Search
3. Depth First Search
4. Depth-limited Search
5. Iterative deepening search
6. Bidirectional search



Informed Strategies

1. Best First Search
2. A* Search

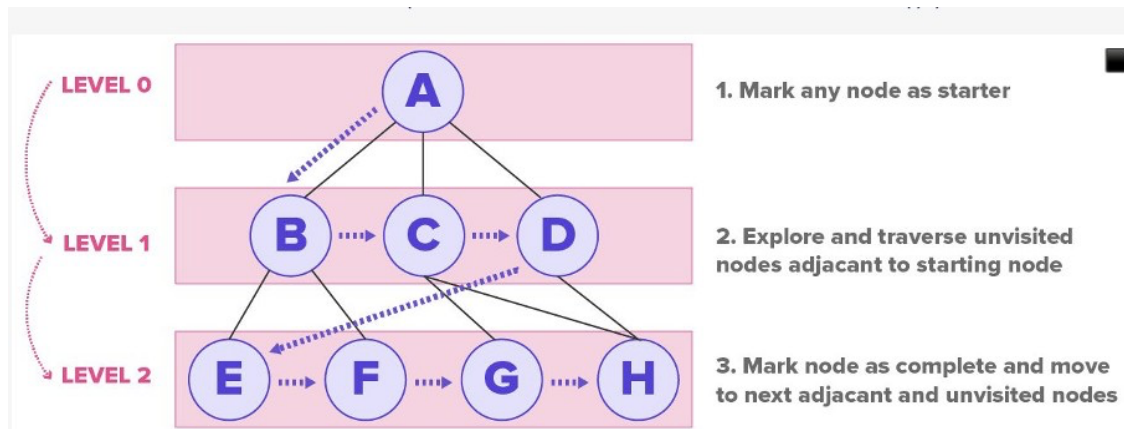


Breadth First Search (BFS)

- Breadth First Search (BFS) is the most common search strategy for traversing a tree or graph.
- This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- Breadth-first search implemented using FIFO queue data structure.
- Advantage
 - If there are more than one solutions for a given problem, then BFS will provide the minimal solution which requires the least number of steps.
- Disadvantages
 - It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
 - BFS needs lots of time if the solution is far away from the root node

BFS (Contd.)

- In BFS, root node is expanded first, then all the successors of the root node are expanded next, then their successors, and so on.
- Breadth-first search can be implemented by calling **TREE-SEARCH** with an empty fringe that is a first-in-first-out (FIFO) queue, assuring that the nodes that are visited first will be expanded first. In other words, calling **TREE-SEARCH(*Problem*, FIFO-QUEUE())** results in a breadth-first search.



■ The FIFO queue puts all newly generated successors at the end of the queue, which means that shallow nodes are expanded before deeper nodes

function BREADTH-FIRST-SEARCH(*problem*) **returns** a solution or failure
return GENERAL-SEARCH(*problem*, ENQUEUE-AT-END)

Depth First Search

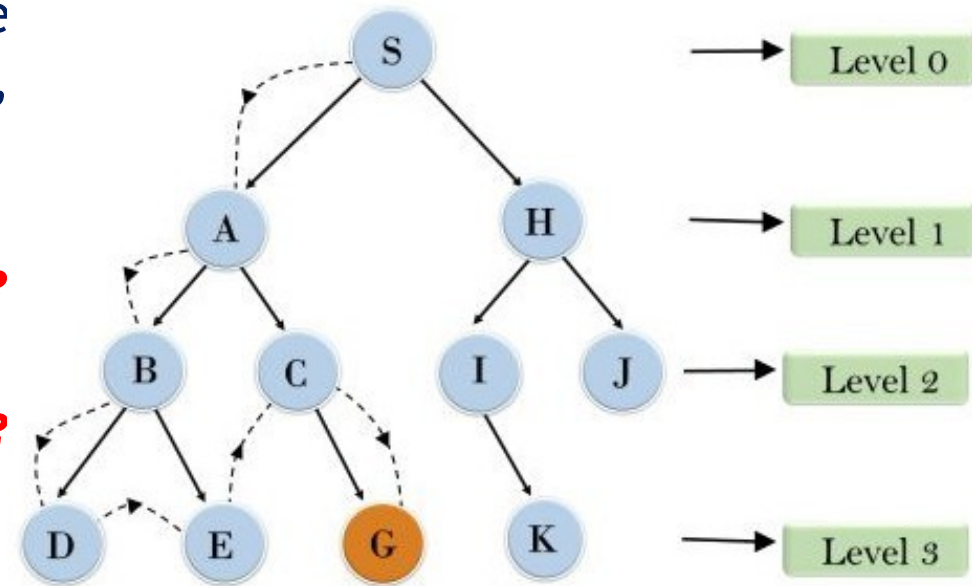
→ Depth First Search

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure
- Depth-first search starts from the root node and follows each path to its greatest depth node before moving to the next path
- DFS uses a stack data structure for its implementation
- Advantage
 - DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
 - It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).
- Disadvantage
 - There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
 - DFS algorithm goes for deep down searching and sometime it may go to the infinite loop

DFS(Contd.)

- It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.
- In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

Root Node → Left Node → Right Node
(or)
Root Node → Right Node → Left Node

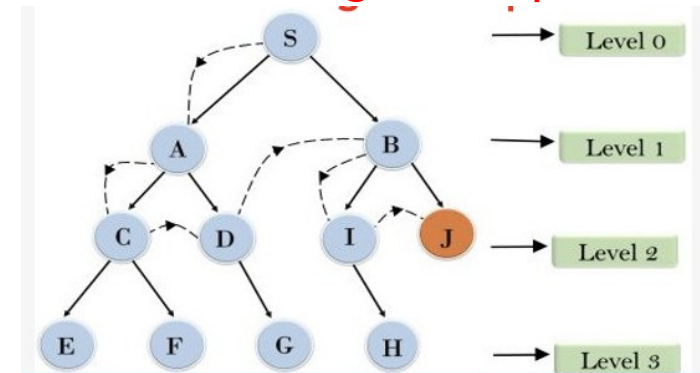


function DEPTH-FIRST-SEARCH(*problem*) **returns** a solution, or failure
 GENERAL-SEARCH(*problem*, ENQUEUE-AT-FRONT)

Depth Limited Search

→ Depth-limited Search

- A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search
- In this algorithm, the node at the depth limit will treat as it has no success nodes further
- Depth-limited search can be terminated with two Conditions of failure
 - **Standard failure value:** It indicates that problem does not have any solution
 - **Cutoff failure value:** It defines no solution for the problem within a given depth limit
- Advantages
 - **Depth-limited search is Memory efficient**
- Disadvantages
 - **It may not be optimal if the problem has more than one solution**



Iterative Deepening Depth First Search

→ Iterative Deepening Depth-First Search

- The iterative deepening algorithm is a *combination of DFS and BFS* algorithms
- This search algorithm finds out the best depth limit and does it by gradually *increasing the limit until a goal is found*
- This algorithm performs depth-first search up to a certain *“Depth Limit”*, and it keeps increasing the depth limit after each iteration until the goal node is found
- This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency
- The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown
- Advantages
 - *It combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency*
- Disadvantages
 - *The main drawback of IDDFS is that it repeats all the work of the previous phase*

Iterative Deepening DFS(Contd)

■ Example

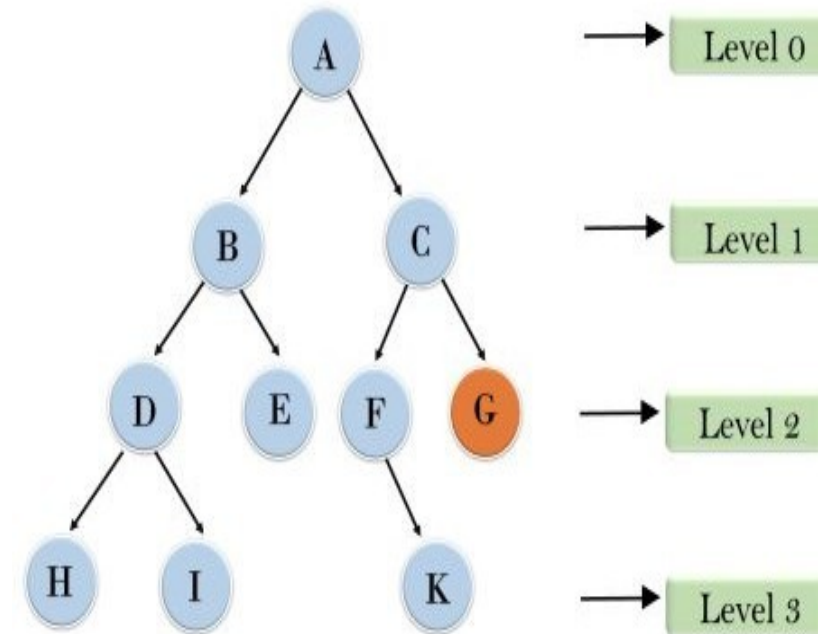
Following tree structure is showing iterative deepening depth-first search. It performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as a

1'st Iteration → A

2'nd Iteration → A, B, C

3'rd Iteration → A, B, D, E, C, F, G

4'th Iteration → A, B, D, H, I, E, C, F, K, G



```

function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution sequence
  inputs: problem, a problem

  for depth ← 0 to ∞ do
    if DEPTH-LIMITED-SEARCH(problem, depth) succeeds then return its result
  end
  return failure
  
```

Credits

1. Artificial Intelligence: A Modern Approach, Stuart J. Russell and Peter Norvig, 4th Edition
2. Artificial Intelligence basics Neeru Gupta and Ramita Mangla
3. Dr. Satish Babu, AIML, RVCE